

Data Preprocessing for Machine Learning

This is a placement-oriented guide to preparing tabular ML data. **Rule zero:** split first; fit every learned preprocessing object only on the training split; then transform validation/test data with that fitted object.

Handling Missing Values

1. Overview

Missing values are absent measurements (**NaN**, **None**, blanks, or sentinels such as **-999**). Most estimators cannot consume them, so they must be understood and handled without distorting the data.

2. Intuition

A blank income is not zero income. Replacing it with a sensible representative value lets a model use the row while an extra flag can tell it that the original value was unknown.

3. Prerequisites

Pandas, data types, mean/median/mode, and train/test splits.

4. Core Concepts

Subtopic	Meaning / example	Interview angle
MCAR	Missing independently of data	Deleting rows is least biased here.
MAR	Missing depends on observed variables	Imputation plus predictors can work.
MNAR	Missing depends on the hidden value	Missingness itself may be informative.
Imputation	Fill median age or mode city	Fit the imputer on train only.
Indicator	age_was_missing is 1 for blanks	Often useful for MNAR-like signals.

5. Algorithm / Working Process

1. Standardize sentinels to **NaN** and measure missingness by column and class.
2. Remove a feature only if it is mostly unusable and not important.
3. Choose median for skewed numeric data, mean for symmetric data, and most-frequent/constant for categories.
4. Fit on train; transform every split.
5. Validate the choice.

6. Mathematical Foundation

Mean imputation uses $\bar{x}_{\text{missing}} = (1/n) \sum x_i$; median imputation uses the 50th percentile. A missingness indicator is $m_i = 1[x_i \text{ is missing}]$. Mean imputation reduces variance, so it can bias coefficients and uncertainty estimates.

7. Practical Implementation

```
import pandas as pd
from sklearn.impute import SimpleImputer

train = pd.DataFrame({'age': [20, None, 50], 'city': ['Delhi', None, 'Pune']})
num = SimpleImputer(strategy='median', add_indicator=True)
cat = SimpleImputer(strategy='most_frequent')
X_num = num.fit_transform(train[['age']])
X_cat = cat.fit_transform(train[['city']])
```

8. Code Explanation

`SimpleImputer` learns the median/mode from `train`; `add_indicator=True` appends a missingness feature. Call `transform`, not `fit_transform`, on validation and test data.

9. Training / Evaluation

Compare imputation strategies through cross-validation using the model metric. Check missingness by target; a random split can hide time-dependent missingness.

10. Complexity and Cost

Simple mean/median fitting is $O(nd)$ time and $O(d)$ learned storage. KNN and iterative imputation cost much more.

11. Common Use Cases

Clinical measurements, customer profiles, sensor outages, optional form fields.

12. Common Mistakes

Treating blanks as zero; imputing before splitting; using a global mean; dropping rows blindly; forgetting categorical missing values.

13. Edge Cases / Limitations

MNAR data cannot be repaired reliably from observed data alone. A column with 95% missing values may be unstable even after imputation.

14. Variations

KNN imputation uses nearby rows; iterative/MICE predicts each feature from others; model-native missing handling (for example, some boosting libraries) learns a missing branch. KNN/MICE matter for projects, not usually first-round interviews.

15. Related Topics

Outliers affect mean imputation; feature scaling affects KNN imputation; data leakage is the main operational risk.

16. Interview Questions

1. **Why median over mean?** Median resists outliers.
2. **Can rows be dropped?** Yes, when few are missing and plausibly MCAR.
3. **Why train-only fitting?** Test statistics leak future information.
4. **What is MCAR?** Missingness independent of all values.
5. **MAR?** Depends on observed values.
6. **MNAR?** Depends on unobserved value.
7. **Why add an indicator?** Missingness can predict the target.
8. **How fill categories?** Mode or explicit "Missing".
9. **Does imputation change variance?** Usually, especially mean imputation.
10. **How select a method?** Cross-validated downstream performance plus domain knowledge.

17. Practice Tasks

Profile nulls in Titanic; compare median and iterative imputation; add indicators; prove that fitting on all rows changes a held-out transform; inspect target rates by missingness.

18. Project Ideas

Project	Stack / data	Resume value
Hospital risk scorer	Pandas, sklearn; diabetes data	Missingness-aware pipeline.
Sensor quality monitor	Pandas; IoT telemetry	Detects outages and imputes gaps.
Loan default model	sklearn; LendingClub-like data	Demonstrates leakage-safe features.

19. Quick Revision

Key idea: preserve usable rows without pretending blanks are real values. Formula: mean/median fill. Use: most tabular data. Trap: fit imputer before split. One-liner: *impute from training data and retain missingness signal when useful*.

20. Final Cheat Sheet

Input/output: incomplete table → complete feature matrix. **Steps:** diagnose, split, train-fit imputer, transform. **Hyperparameters:** strategy, fill value, indicator. **Metric:** downstream CV score. **Pros/cons:** keeps data / can bias distributions. Best for tabular pipelines.

Handling Outliers

1. Overview

Outliers are observations unusually far from the typical distribution. They may be data errors, rare valid events, or the most valuable cases (fraud), so removal is a domain decision.

2. Intuition

If nearly all salaries are ₹3–20 lakh and one is ₹20 crore, mean-based models can be pulled toward it. But a ₹20 crore executive may be valid, not an error.

3. Prerequisites

Percentiles, mean/standard deviation, median/IQR, plots, and domain validation.

4. Core Concepts

Subtopic	Meaning / example	Interview angle
Univariate	One feature is extreme	IQR is robust.
Multivariate	Normal alone, odd jointly	Isolation Forest can help.
IQR rule	Outside $Q1 - 1.5IQR$, $Q3 + 1.5IQR$	Detects, not automatic deletion.
Z-score	Far from mean in standard deviations	Assumes roughly normal distribution.
Capping	Clip at percentile bounds	Keeps row count.

5. Algorithm / Working Process

Visualize distributions; validate units and collection errors; choose a detector; decide to correct, remove, cap, transform, or retain; fit thresholds on train; inspect performance and affected groups.

6. Mathematical Foundation

$IQR = Q3 - Q1$; lower/upper fences are $Q1 - 1.5IQR$, $Q3 + 1.5IQR$. Z-score is $z = (x - \mu) / \sigma$; $|z| > 3$ is a common heuristic, not a law. Log transform uses $\log(1+x)$ for nonnegative heavy tails.

7. Practical Implementation

```
import pandas as pd

def cap_iqr(s):
    q1, q3 = s.quantile([.25, .75]); iqr = q3 - q1
    return s.clip(q1 - 1.5 * iqr, q3 + 1.5 * iqr)

train['income_capped'] = cap_iqr(train['income']) # compute bounds on train only
```

8. Code Explanation

`clip` winsorizes values instead of deleting rows. In production, save the two train-derived bounds and apply those fixed values to new data.

9. Training / Evaluation

Evaluate on realistic held-out data, including rare valid cases. Robust models/tree ensembles often need less treatment than linear regression or distance-based models.

10. Complexity and Cost

Quantiles and clipping are typically $O(n \log n)$ (sorting) per feature; transformations are $O(n)$.

11. Common Use Cases

Transaction fraud, sensor glitches, revenue, medical lab values, price data.

12. Common Mistakes

Deleting every detected outlier; using z-scores on skewed data; computing caps globally; capping the target without justification; ignoring unit errors.

13. Edge Cases / Limitations

Small datasets give unstable quantiles. A multimodal distribution can make valid clusters look anomalous.

14. Variations

Winsorization clips tails; robust scaling uses median/IQR; Isolation Forest isolates rare points; LOF uses local density. IQR and robust scaling are placement essentials.

15. Related Topics

Median imputation, log normalization, robust estimators, anomaly detection, fairness analysis.

16. Interview Questions

1. **Outlier vs anomaly?** An outlier is statistically unusual; anomaly implies an actionable abnormality.
2. **IQR formula?** $Q3 - Q1$.
3. **Why not always remove?** It may be valid signal.
4. **When use z-score?** Roughly Gaussian features.
5. **What is winsorization?** Capping extremes.
6. **Why log transform?** Compresses right tails.
7. **Do trees need scaling?** Usually no, but errors/outliers can still matter.
8. **Multivariate detection?** Isolation Forest/LOF.
9. **Leakage risk?** Learn thresholds on train.
10. **Best response?** Domain check before statistical action.

17. Practice Tasks

Plot boxplots; compare raw/log/capped regression; inject bad units; measure how many rows each rule flags; test Isolation Forest on synthetic anomalies.

18. Project Ideas

Project	Stack / data	Resume value
Card anomaly triage	sklearn; credit-card data	Separates fraud signal from bad data.
Retail price cleaner	Pandas; e-commerce prices	Reproducible capping pipeline.
Sensor dashboard	Pandas/Plotly; telemetry	Explains anomalous readings.

19. Quick Revision

Detect $\neq$ delete. IQR fences are robust; z-score is distribution-sensitive. Fit limits on train. Interview line: *use domain knowledge first, then robust thresholds.*

20. Final Cheat Sheet

Input/output: numeric values $\rightarrow$ validated/capped/transformed values. **Main steps:** inspect, validate, detect, choose action, evaluate. **Parameters:** IQR multiplier/percentile. **Pros/cons:** reduces sensitivity / can erase rare signal. Best for noisy heavy-tailed features.

Encoding Categorical Variables

1. Overview

Encoding maps categories such as city or plan type to numeric features that estimators can use.

2. Intuition

A model cannot infer from the text "Gold"; encoding supplies a numeric representation while preserving the category's meaning.

3. Prerequisites

Feature matrices, nominal versus ordinal variables, and train/test splits.

4. Core Concepts

Subtopic	Meaning / example	Interview angle
Nominal	red, blue, green, no order	One-hot is safe.
Ordinal	low < medium < high	Ordered mapping can preserve order.
Cardinality	Number of unique categories	High cardinality makes one-hot wide.
Unknown category	Value absent during train	Configure safe handling.
Target encoding	Category replaced by target statistic	Must use out-of-fold values.

5. Algorithm / Working Process

Classify each categorical column as nominal/ordinal; count cardinality; choose one-hot, ordinal, frequency, hashing, or leakage-safe target encoding; fit vocabulary on train; define missing/unknown behavior; validate.

6. Mathematical Foundation

An encoding is a function $f: \mathcal{C} \rightarrow \mathbb{R}^k$. One-hot maps category c_j to basis vector e_j ; ordinal maps to an ordered integer. Target encoding estimates $E[y \mid c]$, requiring smoothing and out-of-fold fitting.

7. Practical Implementation

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

preprocess = ColumnTransformer([
    ('city', OneHotEncoder(handle_unknown='ignore'), ['city']),
    ('grade', OrdinalEncoder(categories=[['low', 'medium', 'high']]), ['grade']),
], remainder='passthrough')
X_train_ready = preprocess.fit_transform(X_train)
X_test_ready = preprocess.transform(X_test)
```

8. Code Explanation

`ColumnTransformer` applies a different encoder per column. `handle_unknown='ignore'` makes a new city an all-zero city block rather than a runtime error.

9. Training / Evaluation

Put encoders inside a `Pipeline` so cross-validation refits them per fold. Compare accuracy/F1/AUC and memory; high-cardinality encodings can overfit.

10. Complexity and Cost

Ordinal is $O(n)$ and compact. One-hot stores roughly $O(nk)$ nonzeros for k categories; sparse matrices help.

11. Common Use Cases

Customer region, product type, browser, education level, diagnosis code.

12. Common Mistakes

Using label encoding for nominal inputs; fitting category vocabulary globally; failing on unseen values; target encoding with the same row's target.

13. Edge Cases / Limitations

One-hot becomes huge for IDs. Treating ID-like categories as meaningful categories can memorize training examples.

14. Variations

Frequency encoding uses category count; hashing has fixed width and collisions; learned embeddings suit deep models. One-hot/ordinal are most important for placements.

15. Related Topics

One-hot and label encoding are specific encoders; feature selection can remove sparse dummies; leakage is crucial for target encoding.

16. Interview Questions

1. **Nominal vs ordinal?** Nominal has no order.
2. **Why encode?** Estimators require numbers.
3. **Best default for nominal?** One-hot.
4. **High cardinality?** Frequency/hashing/regularized target encoding or rethink feature.
5. **Unknown category?** Ignore/map safely.
6. **Can integers imply order?** Yes; that is why label encoding can be harmful.
7. **When ordinal encoding?** Genuine ordered levels.
8. **Why pipeline?** Prevent fold leakage.
9. **What is target encoding?** Category's smoothed target mean.
10. **Sparse benefit?** Avoids storing zeros.

17. Practice Tasks

Encode Titanic sex/class; handle a new city at inference; compare ordinal versus one-hot for a linear model; simulate a 10,000-category user ID.

18. Project Ideas

Project	Stack / data	Resume value
Churn classifier	sklearn; Telco churn	Proper mixed-type pipeline.
House-price model	Ames Housing	Handles ordinal quality fields.
Ad-click model	pandas/sklearn; Criteo sample	High-cardinality design choices.

19. Quick Revision

Nominal → one-hot; ordinal → ordered mapping. Fit category set on train. Trap: arbitrary integers create false distance/order.

20. Final Cheat Sheet

Input/output: categories → numeric matrix. **Steps:** identify semantics, choose encoding, train-fit, safe transform. **Parameters:** unknown handling, sparsity, categories. **Pros/cons:** enables models / can widen or leak. Best for all tabular categorical features.

One-Hot Encoding

1. Overview

One-hot encoding creates one binary column per category, making each category independent rather than ordered.

2. Intuition

For `color={red, blue, green}`, red becomes `[1,0,0]`; blue `[0,1,0]`. No color is numerically "larger."

3. Prerequisites

Categorical variables, binary vectors, linear models, and sparse matrices.

4. Core Concepts

Subtopic	Meaning / example	Interview angle
Dummy variable	Binary category column	<code>city_Delhi=1</code> .
Reference drop	Remove one column	Avoids dummy-variable trap in unregularized regression.
Sparse output	Store only ones	Essential for many categories.
Unknown handling	Ignore unseen category	Production safety.

5. Algorithm / Working Process

Learn train categories; allocate one column per category; write 1 for the observed category and 0 elsewhere; optionally drop a baseline; use the same columns at inference.

6. Mathematical Foundation

For category c_i among k , $x_j = 1[c_i=c_j]$; normally $\sum_j x_j=1$. With an intercept, all k columns are linearly dependent because the intercept equals their sum.

7. Practical Implementation

```
from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(handle_unknown='ignore', drop='first', sparse_output=True)
Xtr = ohe.fit_transform(X_train[['city']])
Xte = ohe.transform(X_test[['city']])
print(ohe.get_feature_names_out())
```

8. Code Explanation

`drop='first'` removes one reference category; use it mainly with linear models plus intercept. Trees and regularized models can generally retain all columns.

9. Training / Evaluation

Use pipeline/CV. Watch coefficient stability, sparse matrix width, and unseen-category behavior; evaluate with the task metric.

10. Complexity and Cost

Encoding is $O(n)$ but model cost rises with total dummy columns. Sparse storage has one nonzero per encoded categorical feature per row.

11. Common Use Cases

Low/moderate-cardinality region, device, product category, weekday.

12. Common Mistakes

Refitting on test; dense conversion of huge sparse output; treating all-zero unknown block as a known category; one-hotting unique IDs.

13. Edge Cases / Limitations

Very high cardinality causes memory and overfitting; categories cannot express similarity (Mumbai is no closer to Pune than Delhi).

14. Variations

Drop-first dummy encoding; multi-label multi-hot encoding; binary encoding. For neural nets, embeddings are often preferable at high cardinality.

15. Related Topics

Categorical encoding, logistic regression, feature hashing, embeddings, multicollinearity.

16. Interview Questions

1. **Why use it?** Avoid false ordering.
2. **Dummy trap?** Perfect collinearity with intercept.
3. **Should one column be dropped?** Often for unregularized linear models.
4. **Trees need it?** They need numeric representation but may work well with ordinal/category-native alternatives.
5. **Unknown category?** Ignore safely.
6. **Sparse means?** Store nonzeros only.
7. **High-cardinality problem?** Wide matrix.
8. **Multi-hot?** Multiple categories can be 1.
9. **Fit where?** Training split.
10. **Alternative?** Target/frequency encoding or embeddings.

17. Practice Tasks

Manually encode three colors; compare `drop=None` and `drop='first'`; test new category inference; measure sparse versus dense size.

18. Project Ideas

Project	Stack / data	Resume value
Movie revenue predictor	sklearn; TMDb metadata	Mixed categorical pipeline.
Employee attrition	IBM HR data	Interpretable linear features.
Product recommender baseline	pandas; retail events	Sparse feature engineering.

19. Quick Revision

One category = one binary feature. Formula: indicator vector. Use nominal low-cardinality data. Trap: high-cardinality IDs and train/test column mismatch.

20. Final Cheat Sheet

Input/output: category $\rightarrow k$ binary features. **Parameters:** `drop`, unknown handling, sparse output. **Pros/cons:** safe semantics / wide matrix. **Best:** linear models on nominal columns.

Label Encoding

1. Overview

Label encoding maps labels or categories to integer IDs. In scikit-learn it is primarily intended for target labels (y), not unordered input features.

2. Intuition

`spam`, `ham` can become `1`, `0` because they are output classes. Encoding `red`, `blue`, `green` as `0,1,2` as an input wrongly suggests green is twice red.

3. Prerequisites

Classification targets, nominal/ordinal distinction, and estimator inputs.

4. Core Concepts

Subtopic	Meaning / example	Interview angle
Target label	Class name mapped to ID	Appropriate use of <code>LabelEncoder</code> .
Nominal feature	Arbitrary integer mapping	Usually unsafe for linear/distance models.
Ordinal feature	Explicit ordered IDs	Valid if order is real.
Inverse transform	ID back to original class	Needed for readable predictions.

5. Algorithm / Working Process

Fit the set of target classes on training labels; map each to an integer; train classifier; map predictions back if needed. For ordinal input features, define the order explicitly.

6. Mathematical Foundation

It defines a bijection $f: \mathcal{C} \rightarrow \{0, \dots, K-1\}$. The numerical distances have no semantic meaning unless categories are ordinal.

7. Practical Implementation

```
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder

le = LabelEncoder()
y_train_id = le.fit_transform(y_train)      # correct target use
y_pred_name = le.inverse_transform([0, 1])
size = OrdinalEncoder(categories=[['S', 'M', 'L', 'XL']]) # ordered input
```

8. Code Explanation

LabelEncoder learns target class names. **OrdinalEncoder** is clearer for feature columns and lets you declare the business order.

9. Training / Evaluation

Classification metrics accept encoded labels, but preserve class names for reports. Ensure a validation split has all important classes when possible.

10. Complexity and Cost

Fit and transform are approximately $O(n \log K)$ / $O(n)$ with tiny $O(K)$ mapping memory.

11. Common Use Cases

Binary/multiclass target labels; ordered satisfaction or education levels.

12. Common Mistakes

Label-encoding nominal input features; assuming class 2 is “better” than class 1; failing on a new class; deriving arbitrary order from alphabetical order.

13. Edge Cases / Limitations

Unseen labels at evaluation need a policy. Integer-valued categories may be mistaken for continuous values by some models.

14. Variations

Binary encoding compresses categories; ordinal encoding declares order; embeddings learn representations. Target encoding is different: it uses **y** and can leak.

15. Related Topics

One-hot encoding for nominal inputs; classification class mapping; ordinal regression.

16. Interview Questions

1. **What is label encoding?** Category-to-integer mapping.
2. **Best use?** Target labels.
3. **Why risky for city?** Invents order/distance.
4. **When valid for feature?** True ordinal scale.
5. **LabelEncoder vs OrdinalEncoder?** Target versus input feature API.
6. **Can trees tolerate IDs?** Sometimes, but split order is still artificial.
7. **Inverse transform?** Restore class text.
8. **Unknown class?** Handle explicitly.
9. **Does it increase dimensions?** No.
10. **Alternative to nominal input?** One-hot.

17. Practice Tasks

Encode a three-class target; decode predictions; show a linear model’s false trend after encoding colors; encode ordered ratings correctly.

18. Project Ideas

Project	Stack / data	Resume value
News classifier	sklearn; AG News	Clean target label mapping.
Review rating predictor	pandas/sklearn; Amazon reviews	Uses ordinal ratings responsibly.
Ticket routing	sklearn; support tickets	Decodes human-readable predictions.

19. Quick Revision

Use for **y**, not arbitrary nominal **x**. No formula beyond a class-ID map. Trap: artificial order. Interview line: *integer encoding is semantic only when the categories are ordered.*

20. Final Cheat Sheet

Input/output: class names → IDs. **Steps:** train-fit map, transform labels, train, inverse-transform. **Pros/cons:** compact / false order for features. **Best:** classification targets.

Feature Scaling

1. Overview

Feature scaling puts numeric features on comparable scales so magnitude does not dominate models based on distance, dot products, or gradients.

2. Intuition

If income ranges to 1,000,000 and age to 80, Euclidean distance mostly measures income unless the features are scaled.

3. Prerequisites

Mean, standard deviation, min/max, gradients, distance, and train/test split.

4. Core Concepts

Subtopic	Meaning / example	Interview angle
Scale-sensitive models	KNN, SVM, k-means, PCA, neural nets	Usually need scaling.
Scale-insensitive models	Decision trees/random forests	Split order is unchanged.
Standardization	Zero mean, unit variance	Default common choice.
Normalization	Bounded range or unit norm	Clarify which meaning is meant.
Robust scaling	Median/IQR scaling	Useful with outliers.

5. Algorithm / Working Process

Identify numeric features and model; split; fit scaler on training columns; transform all splits; retain the scaler with the model; do not scale target unless the task needs it.

6. Mathematical Foundation

Common transforms: standard score $z = (x - \mu) / \sigma$; min-max $x' = (x - \min) / (\max - \min)$; robust score $(x - \text{median}) / \text{IQR}$. Scaling changes optimization geometry but not information.

7. Practical Implementation

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

8. Code Explanation

The pipeline fits `StandardScaler` only when `.fit(X_train, y_train)` runs and applies identical train statistics at prediction time.

9. Training / Evaluation

Compare scaled/unscaled baselines for scale-sensitive models. Cross-validation pipelines prevent per-fold leakage; convergence speed is also a useful diagnostic.

10. Complexity and Cost

Most scalers require $O(nd)$ fit and transform, $O(d)$ learned state. They are CPU-cheap relative to training.

11. Common Use Cases

KNN similarity search, SVM, logistic/linear regression with regularization, PCA, k-means, neural networks.

12. Common Mistakes

Scaling before split; scaling one-hot columns unnecessarily; applying standardization blindly to tree-only models; failing to persist the scaler.

13. Edge Cases / Limitations

Min-max is fragile to future values outside train range. `StandardScaler` is sensitive to extreme outliers.

14. Variations

Standard, min-max, max-abs, robust, L2 row normalization. Standard/robust are key placement knowledge.

15. Related Topics

Standardization and normalization are specific scalers; outlier handling determines robust choice; PCA/SVM/KNN are scale-sensitive.

16. Interview Questions

1. **Why scale?** Prevent magnitude domination.
2. **Which models need it?** KNN/SVM/k-means/PCA/gradient-based models.
3. **Trees?** Usually not.
4. **Fit where?** Train only.
5. **Does scaling remove outliers?** No.
6. **Which for outliers?** RobustScaler.
7. **Does it change ranks?** Monotonic scaling does not.
8. **What about one-hot?** Usually leave as 0/1.
9. **Why help gradient descent?** Better-conditioned loss surface.
10. **Persist what?** Fitted scaler plus model.

17. Practice Tasks

Run KNN with/without scaling; compare StandardScaler/RobustScaler on injected outliers; inspect SVM convergence; package scaler and model in a pipeline.

18. Project Ideas

Project	Stack / data	Resume value
Customer clustering	sklearn; mall customers	Shows geometric preprocessing.
Credit risk	sklearn; German Credit	Regularized model pipeline.
Digit PCA + classifier	sklearn; digits	Connects scaling to PCA.

19. Quick Revision

Scale-sensitive model → scale numeric train features. Formula: z-score/min-max. Trap: leakage. One-liner: *scaling changes units, not information, but it changes model geometry.*

20. Final Cheat Sheet

Input/output: numeric columns → comparable numeric columns. **Parameters:** scaler type. **Metrics:** downstream score/convergence. **Pros/cons:** stable optimization / transform can be outlier-sensitive. Best for distance and gradient models.

Standardization

1. Overview

Standardization rescales each feature to approximately zero mean and unit standard deviation.

2. Intuition

It converts marks in 0–100 and income in lakhs into “how many standard deviations from typical?” so both are comparable.

3. Prerequisites

Mean, variance, standard deviation, and feature scaling.

4. Core Concepts

Subtopic	Meaning / example	Interview angle
Centering	Subtract training mean	Result mean is zero on train.
Scaling	Divide by training standard deviation	Result variance is one on train.
Zero variance	Same value for all rows	Cannot carry predictive variation.
Robust alternative	Median/IQR instead	Better with heavy outliers.

5. Algorithm / Working Process

For each train numeric column calculate μ , σ ; replace values by $(x-\mu)/\sigma$; reuse those exact values for validation, test, and production rows.

6. Mathematical Foundation

$z=(x-\mu_{\text{train}})/\sigma_{\text{train}}$, where $\sigma=\sqrt{(1/n)\sum(x_i-\mu)^2)}$ in common scaler implementations. Standardization does not guarantee a Gaussian distribution.

7. Practical Implementation

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_z = scaler.fit_transform(X_train[['age', 'income']])
X_test_z = scaler.transform(X_test[['age', 'income']])
```

8. Code Explanation

`fit_transform` estimates and applies train statistics; `transform` applies them unchanged. The transformed test mean need not be zero—and that is correct.

9. Training / Evaluation

Useful before SVM, KNN, PCA, logistic regression, and neural models. Check cross-validated metric and convergence; inspect outliers first.

10. Complexity and Cost

Two passes per feature: $O(nd)$ time, $O(d)$ memory.

11. Common Use Cases

Regularized regression, PCA, k-means, KNN, standardized neural-network inputs.

12. Common Mistakes

Expecting values in `[0,1]`; using test mean; scaling target labels; assuming it normalizes a skewed distribution.

13. Edge Cases / Limitations

Outliers inflate `σ`; constant features need removal or special handling; sparse matrices may require `with_mean=False`.

14. Variations

RobustScaler (median/IQR), MaxAbsScaler (sparse data), whitening (also decorrelates). Whitening is more advanced than typical placements.

15. Related Topics

Normalization bounds/range differs; PCA and regularization depend on comparable scale; robust scaling handles outliers.

16. Interview Questions

1. **Formula?** $(x-\mu)/\sigma$.
2. **Range?** Unbounded.
3. **Does it make data normal?** No.
4. **Why train mean?** Prevent leakage.
5. **Test mean zero?** Not necessarily.
6. **Outlier issue?** Mean/std are sensitive.
7. **Trees need it?** Usually no.
8. **Zero variance?** Drop/handle feature.
9. **Sparse centering?** Can destroy sparsity.
10. **Alternative?** Robust scaling.

17. Practice Tasks

Calculate z-scores by hand; verify train column mean/variance; compare PCA results with and without it; use RobustScaler on salary data.

18. Project Ideas

Project	Stack / data	Resume value
Diabetes regressor	sklearn; diabetes	Regularization-ready pipeline.
Customer segments	sklearn; retail data	Correct k-means geometry.
Fraud SVM	sklearn; card data	Handles numeric scale disparity.

19. Quick Revision

$z=(x-\mu_{\text{train}})/\sigma_{\text{train}}$; zero mean/unit variance on train; use for SVM/KNN/PCA. Trap: it is not min-max normalization.

20. Final Cheat Sheet

Input/output: numeric feature → z-score. **Parameters:** mean/std learned on train. **Pros/cons:** comparable units / outlier-sensitive. **Best:** gradient, distance, covariance methods.

Normalization

1. Overview

Normalization commonly means min-max scaling to a fixed interval, though in ML it can also mean L1/L2 normalizing each sample. State which definition you mean.

2. Intuition

Min-max turns a train feature from 20–80 into 0–1. L2 normalization turns a document's count vector into a direction of length one.

3. Prerequisites

Minimum/maximum, vector norms, feature scaling.

4. Core Concepts

Subtopic	Meaning / example	Interview angle
Min-max	Column-wise bounded scaling	Common tabular meaning.
L1 norm	Row absolute values sum to one	Useful for proportions.
L2 norm	Row Euclidean norm is one	Common for cosine similarity.
Clipping	Bound unseen values	Policy decision, not default math.

5. Algorithm / Working Process

For min-max, learn train min/max per column and transform each value. For vector normalization, compute a norm per row and divide that row; choose based on whether absolute magnitude should be retained.

6. Mathematical Foundation

Min-max: $x' = (x - \min_{\text{train}}) / (\max_{\text{train}} - \min_{\text{train}})$. L2: $x' = x / \|x\|_2$, $\|x\|_2 = \sqrt{\sum x_j^2}$. L1: $x' = x / \sum |x_j|$.

7. Practical Implementation

```
from sklearn.preprocessing import MinMaxScaler, Normalizer

mm = MinMaxScaler(feature_range=(0, 1))
X_train_mm = mm.fit_transform(X_train[['age', 'income']])
X_test_mm = mm.transform(X_test[['age', 'income']])
unit_rows = Normalizer(norm='l2').fit_transform([[3, 4], [1, 1]])
```

8. Code Explanation

MinMaxScaler scales columns using train limits. **Normalizer** scales rows independently and usually needs no learned training statistics.

9. Training / Evaluation

Use min-max for bounded inputs/neural nets when suitable; use L2 for TF-IDF/cosine workflows. Compare metrics and confirm future values are handled sensibly.

10. Complexity and Cost

All common variants are $O(nd)$ time. Min-max stores two values per feature; row normalization needs no fitted state.

11. Common Use Cases

Pixel values, bounded neural features, TF-IDF vectors, proportions, similarity search.

12. Common Mistakes

Calling min-max and L2 the same operation; using global min/max; allowing leakage; assuming test values always stay in $[0,1]$.

13. Edge Cases / Limitations

An outlier compresses ordinary min-max values; a zero vector cannot be normalized to unit length; unseen values can exceed the training range.

14. Variations

Min-max to $[-1,1]$; MaxAbs scaling; L1/L2 normalization; quantile transform. Min-max vs standardization is a frequent interview question.

15. Related Topics

Feature scaling, standardization, cosine similarity, image preprocessing, outlier handling.

16. Interview Questions

1. **Min-max formula?** $(x - \min) / (\max - \min)$.
2. **Range?** Usually $[0,1]$.
3. **Standardization difference?** Z-score is unbounded and mean-centered.
4. **L2 normalization?** Unit row length.
5. **Does it remove outliers?** No.
6. **Why use on images?** Stable bounded input.
7. **Fit min/max where?** Train only.
8. **Future larger value?** It may exceed 1.
9. **Zero vector?** Keep zero/handle safely.
10. **Cosine use?** L2-normalized vectors make dot product cosine similarity.

17. Practice Tasks

Scale values manually; show an outlier's compression effect; L2-normalize TF-IDF; compare image classifier input conventions.

18. Project Ideas

Project	Stack / data	Resume value
Image digit baseline	sklearn; digits	Pixel preprocessing awareness.
Document similarity	sklearn; 20 Newsgroups	L2/cosine retrieval.
Fitness dashboard	pandas/sklearn	Bounded feature pipeline.

19. Quick Revision

Min-max is column range scaling; L2 is row length scaling. Trap: min-max is outlier-sensitive and test values can leave the range.

20. Final Cheat Sheet

Input/output: values/vectors → bounded or unit-norm representation. **Parameters:** range, norm. **Pros/cons:** intuitive range / outlier-sensitive. **Best:** pixels and cosine-based text vectors.

Train/Validation/Test Split

1. Overview

Splitting reserves independent data for model selection and final unbiased performance estimation.

2. Intuition

Training is studying past exam questions; validation is choosing how to study; test is the sealed final exam used once.

3. Prerequisites

Supervised learning, metrics, random sampling, and distribution shift.

4. Core Concepts

Subtopic	Meaning / example	Interview angle
Train	Learns parameters	Largest split.
Validation	Tunes choices/thresholds	Never used to fit final claims.
Test	Final one-time evaluation	Keep sealed.
Stratification	Preserve class ratios	Important for imbalanced classification.
Group/time split	Keep related/future rows apart	Prevents realistic leakage.

5. Algorithm / Working Process

Define the deployment unit/time; reserve test first; split remaining train/validation (or cross-validate); stratify/group/time-order as appropriate; fit preprocessing/model only on train; tune on validation; refit

chosen pipeline on train+validation if justified; evaluate test once.

6. Mathematical Foundation

Estimated generalization is $R_{\text{test}} = (1/n_{\text{test}}) \sum L(y_i, \hat{y}_i)$. Random i.i.d. splits estimate future i.i.d. performance; time/group dependence violates that assumption.

7. Practical Implementation

```
from sklearn.model_selection import train_test_split

X_tv, X_test, y_tv, y_test = train_test_split(X, y, test_size=.20,
                                              stratify=y, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_tv, y_tv, test_size=.25,
                                                  stratify=y_tv,
                                                  random_state=42)
# Result: 60% train, 20% validation, 20% test
```

8. Code Explanation

The second split takes 25% of the 80% temporary set, yielding 20% total validation. `stratify` preserves class balance.

9. Training / Evaluation

Tune hyperparameters only on validation/CV. Report final test metrics with confidence intervals where possible. Use `TimeSeriesSplit` for time and group-aware splitters for repeated entities.

10. Complexity and Cost

Split creation is $O(n)$; validation/CV multiplies training cost by number of folds/configurations.

11. Common Use Cases

Any supervised project; customer churn, medical outcomes, recommender events, forecasting.

12. Common Mistakes

Using test repeatedly; random-splitting same user across sets; scaling before split; non-stratified split with rare classes; shuffling time series.

13. Edge Cases / Limitations

Tiny data makes a fixed validation set noisy; use cross-validation. Future distribution can differ even with a perfect split.

14. Variations

K-fold CV, stratified K-fold, group K-fold, nested CV, rolling time split. Nested CV matters for rigorous research/model comparison.

15. Related Topics

Data leakage, imbalanced datasets/stratification, hyperparameter tuning, cross-validation.

16. Interview Questions

1. **Why three splits?** Separate fitting, tuning, final evaluation.
2. **Typical ratios?** 60/20/20 or 70/15/15, data-dependent.
3. **Why stratify?** Preserve class ratio.
4. **Can test tune thresholds?** No.
5. **Small data?** CV.
6. **Time series?** Chronological split.
7. **Repeated customer rows?** Group split.
8. **Fit scaler on?** Train.
9. **Nested CV?** Unbiased tuning comparison.
10. **Test once?** To avoid optimistic selection.

17. Practice Tasks

Create stratified split; compare class ratios; show customer leakage under random split; implement chronological split; run 5-fold CV.

18. Project Ideas

Project	Stack / data	Resume value
Churn predictor	sklearn; Telco churn	Reproducible evaluation protocol.
Sales forecast	pandas/sklearn; retail time series	Time-aware evaluation.
Patient readmission	sklearn; hospital data	Group-aware clinical split.

19. Quick Revision

Train learns, validation chooses, test reports. Split before preprocessing. Trap: same entity/future information in train and test.

20. Final Cheat Sheet

Input/output: labeled rows → independent partitions. **Parameters:** ratios, random seed, stratify/group/time. **Metric:** held-out generalization. **Pros/cons:** honest estimate / less training data. Best for every ML workflow.

Data Leakage

1. Overview

Data leakage occurs when training uses information unavailable at the real prediction time, producing deceptively high validation/test scores.

2. Intuition

Predicting loan default using a field created after default is like receiving the answer sheet before the exam.

3. Prerequisites

Splits, feature engineering, preprocessing fitting, and deployment timeline.

4. Core Concepts

Subtopic	Meaning / example	Interview angle
Target leakage	Feature directly/indirectly reveals label	<code>refund_issued</code> predicts churn.
Train-test contamination	Test statistics affect training	Global scaling/imputation.
Temporal leakage	Future data used for past prediction	Tomorrow's sales in features.
Group leakage	Same person appears across splits	Memorization masquerades as skill.
Pipeline	Fits transforms per train fold	Primary defense.

5. Algorithm / Working Process

Write the prediction timestamp and available data contract; split using the deployment boundary; audit every feature's source/time; build a pipeline; use CV consistent with deployment; investigate suspiciously high metrics; test on a later/held-out cohort.

6. Mathematical Foundation

Leakage violates the required conditional information set: prediction should be $\hat{y}_t = f(X_{\leq t})$, not $f(X_{\leq t}, X_{> t}, y)$. Reported empirical risk then underestimates deployment risk.

7. Practical Implementation

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([('impute', SimpleImputer(strategy='median')),
                  ('scale', StandardScaler()), ('model', LogisticRegression())])
pipe.fit(X_train, y_train) # each learned step sees training rows only
```

8. Code Explanation

Pipelines ensure a cross-validation fold fits imputation and scaling on that fold's training portion—not the validation portion.

9. Training / Evaluation

Compare random and time/group holdouts; a dramatic drop often signals leakage or shift. Inspect features with implausibly high correlation/AUC.

10. Complexity and Cost

Leakage prevention costs little computationally; the real cost is feature lineage and correct split design.

11. Common Use Cases

Credit scoring, healthcare, fraud, forecasting, recommender systems, any production model.

12. Common Mistakes

Preprocessing full data; target encoding without folds; selecting features on all rows; duplicate rows across sets; post-outcome features; using future aggregates.

13. Edge Cases / Limitations

Some legitimate features are delayed or revised; the deployment data contract, not intuition, decides availability.

14. Variations

Look-ahead bias in finance, label leakage, preprocessing leakage, and evaluation leakage. Temporal leakage is especially important in interviews.

15. Related Topics

Splits, cross-validation, target encoding, feature selection, MLOps feature stores.

16. Interview Questions

1. **Define leakage.** Future/unavailable information enters training.
2. **Scaler leakage?** Fit scaler on all rows.
3. **Feature selection leakage?** Select using test/whole data.
4. **Target leakage example?** Post-outcome status.
5. **Why pipeline?** Train-fold-only transforms.
6. **Time-series defense?** Chronological validation.
7. **Group leakage?** Same entity in both sets.
8. **Why dangerous?** Offline score is falsely optimistic.
9. **Can high accuracy signal it?** Yes, investigate.
10. **Best prevention?** Define prediction-time availability first.

17. Practice Tasks

Deliberately leak a post-outcome column and observe AUC; fix global scaling with a pipeline; create group/time split; audit a feature table by timestamp.

18. Project Ideas

Project	Stack / data	Resume value
Leakage audit notebook	pandas/sklearn	Demonstrates production judgment.
Transaction fraud model	sklearn; temporal transactions	Uses cutoff-safe aggregates.
Churn feature registry	pandas/MLflow	Documents feature availability.

19. Quick Revision

Only use information available when prediction is made. Pipeline plus realistic split. Trap: global preprocessing and post-event fields.

20. Final Cheat Sheet

Definition: contamination with unavailable information. **Defense:** time contract, correct split, train-fit pipelines. **Metric clue:** suspiciously high offline score. **Con:** invalidates evaluation. **Best use:** audit every model.

Imbalanced Datasets

1. Overview

An imbalanced dataset has unequal class frequencies, such as 1% fraud and 99% normal. Accuracy can look excellent while the minority class is missed entirely.

2. Intuition

Always predicting "not fraud" gets 99% accuracy on 1% fraud data but catches no fraud.

3. Prerequisites

Binary classification, confusion matrix, precision, recall, class distributions.

4. Core Concepts

Subtopic	Meaning / example	Interview angle
Majority/minority	Frequent/rare label	Minority is not always positive.
Precision	Correct among positive predictions	Controls false alarms.
Recall	Found among actual positives	Controls misses.
PR-AUC	Precision-recall tradeoff area	More informative than ROC-AUC when rare.
Class weight	Penalize minority errors more	Strong baseline.

5. Algorithm / Working Process

Inspect counts and business costs; stratify split; choose precision/recall-oriented metric; build class-weight baseline; tune threshold on validation; consider resampling only within training folds; evaluate per-class and calibration.

6. Mathematical Foundation

$\text{precision} = TP / (TP + FP)$, $\text{recall} = TP / (TP + FN)$, $F1 = 2PR / (P + R)$, $\text{balanced accuracy} = (TPR + TNR) / 2$. A weighted loss can use $L = \sum w_y \ell(y, \hat{y})$.

7. Practical Implementation

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, average_precision_score

clf = LogisticRegression(class_weight='balanced', max_iter=1000).fit(X_train,
y_train)
p = clf.predict_proba(X_val)[:, 1]
print(average_precision_score(y_val, p))
print(classification_report(y_val, p >= .30)) # threshold chosen on validation
```

8. Code Explanation

`class_weight='balanced'` increases minority error cost. Probability threshold `.30` is a business/model-selection choice, never an automatic default.

9. Training / Evaluation

Use stratified CV; report PR-AUC, recall, precision, F1, confusion matrix, and cost. Do not report accuracy alone. Calibrate probabilities before cost-based thresholding if needed.

10. Complexity and Cost

Weights have negligible cost. Over/under-sampling changes effective training rows; synthetic methods cost additional neighbor search.

11. Common Use Cases

Fraud, rare disease screening, manufacturing defects, security alerts, churn.

12. Common Mistakes

Accuracy-only reporting; resampling validation/test data; oversampling before CV split; ignoring threshold; using random split without stratification.

13. Edge Cases / Limitations

Extremely few positives make metrics high-variance. The real deployment prevalence may differ from training prevalence.

14. Variations

Undersampling, oversampling, class weights, SMOTE, focal loss, anomaly detection. Class weights and threshold tuning should be tried first.

15. Related Topics

SMOTE, stratified split, calibration, ROC-AUC vs PR-AUC, cost-sensitive learning.

16. Interview Questions

1. **Why accuracy fails?** Majority prediction dominates it.
2. **Precision formula?** $TP / (TP + FP)$.
3. **Recall formula?** $TP / (TP + FN)$.
4. **When prioritize recall?** Costly missed positives.
5. **When precision?** Costly false alarms.
6. **Why PR-AUC?** Focuses on positives.
7. **First remedy?** Class weight + threshold.
8. **Oversample test?** Never.
9. **Why stratify?** Preserve rare class ratio.
10. **Can ROC-AUC mislead?** Yes, it can look good with very rare positives.

17. Practice Tasks

Train fraud baseline; compare accuracy and PR-AUC; tune threshold to recall target; try weights; inspect confusion matrices at three thresholds.

18. Project Ideas

Project	Stack / data	Resume value
Fraud detector	sklearn; Credit Card Fraud	Cost-aware evaluation.
Disease screening	sklearn; UCI health data	Recall/precision tradeoff.
Defect classifier	PyTorch/sklearn; industrial images	Imbalance-aware training.

19. Quick Revision

Rare classes demand precision/recall/PR-AUC, not accuracy. Stratify and resample training only. Trap: 99% accuracy can equal zero useful detections.

20. Final Cheat Sheet

Input/output: skewed labels → cost-aware classifier. **Steps:** inspect, stratify, weight/tune/resample train, evaluate PR metrics. **Parameters:** weights, threshold. **Pros/cons:** captures rare cases / tradeoffs false alarms. Best for rare-event classification.

SMOTE Basics

1. Overview

SMOTE (Synthetic Minority Over-sampling Technique) creates synthetic minority examples by interpolating between minority neighbors instead of merely duplicating them.

2. Intuition

Between two similar minority points, SMOTE places a new point partway along the line connecting them. It expands the minority region.

3. Prerequisites

Imbalanced classification, KNN, Euclidean distance, feature scaling, and train-only resampling.

4. Core Concepts

Subtopic	Meaning / example	Interview angle
Minority neighbors	Nearest minority points	Scaling affects which are neighbors.
Interpolation	New point between two points	Not a copy.
<code>k_neighbors</code>	Neighbors considered	Needs enough minority samples.
Sampling strategy	Desired minority proportion	Avoid blindly forcing 1:1.
SMOTENC	Handles categorical columns	Plain SMOTE is numeric-space method.

5. Algorithm / Working Process

Within each training fold only: scale/prepare numeric data; choose a minority point and one of its `k` minority neighbors; sample $\lambda \in [0, 1]$; create synthetic point; train classifier; validate on untouched original validation data.

6. Mathematical Foundation

For minority point x_i , neighbor x_{nn} , and $\lambda \sim \text{Uniform}(0, 1)$, $x_{\text{new}} = x_i + \lambda(x_{nn} - x_i)$. Neighbor distances are normally Euclidean, hence feature scaling matters.

7. Practical Implementation

```
# pip install imbalanced-learn
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([('scale', StandardScaler()), ('smote', SMOTE(random_state=42)),
                ('model', LogisticRegression(max_iter=1000))])
pipe.fit(X_train, y_train) # SMOTE acts only during fit
```

8. Code Explanation

`imblearn.pipeline.Pipeline` is important: its sampler runs on training folds during fitting but does not synthesize validation/test examples. Do not use sklearn's pipeline for samplers.

9. Training / Evaluation

Use stratified CV and PR-AUC/recall/precision. Compare against class weights. Tune `sampling_strategy` and `k_neighbors`; inspect whether synthetic samples cross class boundaries.

10. Complexity and Cost

Nearest-neighbor search costs roughly $O(n_{\min} d)$ per query (more without indexing); memory grows with the number of generated rows.

11. Common Use Cases

Moderately imbalanced numeric tabular data: fraud, risk, defects, medical classification.

12. Common Mistakes

SMOTE before split; SMOTE on test data; using unscaled features; applying plain SMOTE to one-hot/categorical data; assuming synthetic samples are real evidence.

13. Edge Cases / Limitations

It can create ambiguous points near overlap/noise, amplify minority outliers, and fails when there are fewer minority rows than neighbors. It is unsuitable for temporal order without care.

14. Variations

Borderline-SMOTE focuses near boundaries; ADASYN favors hard regions; SMOTENC handles mixed data; SMOTEENN combines oversampling and cleaning. Basics and leakage rules matter most in placements.

15. Related Topics

Imbalanced datasets, KNN/scaling, class weights, undersampling, decision thresholds.

16. Interview Questions

1. **What does SMOTE do?** Interpolates minority neighbors.
2. **Formula?** $x + \lambda(x_{nn} - x)$.
3. **Why train only?** Synthetic information can leak validation/test structure.
4. **Why scale?** Distance chooses neighbors.
5. **Does it duplicate?** No, usually interpolates.
6. **Can it handle categories?** Use SMOTENC or alternatives.
7. **Main risk?** Noisy/boundary synthetic points.
8. **Alternative?** Class weights.
9. **Tune what?** $k_{\text{neighbors}}$, ratio.
10. **Metric?** PR-AUC/recall/precision, not accuracy alone.

17. Practice Tasks

Visualize 2D synthetic points; compare weights vs SMOTE; intentionally leak by resampling before split; tune $k_{\text{neighbors}}$; test SMOTENC on mixed data.

18. Project Ideas

Project	Stack / data	Resume value
Fraud comparison study	imbalanced-learn; card fraud	Proper sampler CV pipeline.
Disease risk model	sklearn; health data	Recall-oriented design.
Defect prediction	Pandas/sklearn; manufacturing data	Explains resampling limits.

19. Quick Revision

SMOTE interpolates minority neighbors in train folds only. Scale first for numeric distance. Trap: oversampling before split leaks.

20. Final Cheat Sheet

Input/output: minority numeric samples → augmented training set. **Parameters:** ratio, `k_neighbors`. **Metric:** PR-AUC/recall. **Pros/cons:** reduces duplication / can synthesize noise. **Best:** clean numeric imbalance.

Feature Selection

1. Overview

Feature selection chooses a useful subset of input columns, reducing noise, training cost, and overfitting while improving interpretability.

2. Intuition

When predicting house price, area and location may help; a random row ID mostly adds distraction. Selection keeps signal, not every available column.

3. Prerequisites

Features/targets, correlation, cross-validation, regularization, and model metrics.

4. Core Concepts

Subtopic	Meaning / example	Interview angle
Filter	Score feature independent of model	Fast; <code>SelectKBest</code> .
Wrapper	Search subsets using model score	Expensive; RFE.
Embedded	Model selects while training	L1, tree importance.
Redundancy	Duplicate correlated signals	Can destabilize linear coefficients.
Leakage	Selecting on all rows	Selection must be inside CV pipeline.

5. Algorithm / Working Process

Remove obvious IDs/leakage features with domain knowledge; choose filter/wrapper/embedded method; fit selector inside train/CV folds; tune `k` or strength on validation; retrain selected pipeline; assess stability and business plausibility.

6. Mathematical Foundation

Filter examples include correlation and mutual information $I(X;Y) = \sum p(x,y) \log[p(x,y)/(p(x)p(y))]$. L1 regularization minimizes $\text{loss} + \lambda \sum |w_j|$, driving some coefficients exactly to zero. RFE repeatedly removes least useful features.

7. Practical Implementation

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_selection import SelectKBest, mutual_info_classif
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = make_pipeline(StandardScaler(), SelectKBest(mutual_info_classif, k=10),
                      LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

8. Code Explanation

`SelectKBest` scores only the training data inside the pipeline, keeps the top ten columns, and sends them to logistic regression. Tune `k` through CV rather than choosing it from test results.

9. Training / Evaluation

Measure held-out metric, feature count, latency, and selection stability across folds. Importance is not causality; correlated features can share or hide importance.

10. Complexity and Cost

Filters are often $O(nd)$; RFE retrains models repeatedly and is expensive; embedded L1/tree selection approximately costs a model fit.

11. Common Use Cases

High-dimensional text/genomics, interpretable credit models, latency-limited services, noisy tabular datasets.

12. Common Mistakes

Selecting on full data; using correlation alone for nonlinear relationships; treating tree importance as causal; retaining IDs; ignoring correlated-feature instability.

13. Edge Cases / Limitations

A weak feature may become useful only in interaction; filter methods can miss that. Correlated features make any one selected set unstable.

14. Variations

Variance threshold, univariate tests/mutual information, RFE, L1/Lasso, tree-based selection, sequential selection, PCA (feature extraction—not selection). Filter/L1/RFE are placement essentials.

15. Related Topics

Regularization, PCA, multicollinearity, feature engineering, data leakage, model interpretability.

16. Interview Questions

1. **Why select features?** Less noise/cost and better interpretability.
2. **Filter/wrapper/embedded?** Score-alone/search-with-model/model-internal.
3. **L1 effect?** Drives coefficients to zero.
4. **RFE?** Iteratively remove weakest model feature.
5. **PCA selection?** No; it creates new components.
6. **Leakage risk?** Selection before CV.
7. **Correlation limitation?** Misses nonlinear/interaction signal.
8. **Tree importance causal?** No.
9. **High cardinality IDs?** Usually remove.
10. **How choose k?** Cross-validation plus latency/interpretability constraints.

17. Practice Tasks

Compare all features versus top-k; select within/ outside pipeline to observe leakage; run L1 logistic regression; test stability across folds; compare PCA and selection.

18. Project Ideas

Project	Stack / data	Resume value
Gene expression classifier	sklearn; OpenML genomics	High-dimensional selection.
Credit scoring	sklearn; German Credit	Compact interpretable model.
Spam detector	sklearn; SMS Spam	Sparse feature reduction.

19. Quick Revision

Select useful original columns, inside CV. Filter is fast, wrapper expensive, embedded learns selection. Trap: feature selection on all data leaks labels.

20. Final Cheat Sheet

Input/output: many original features → selected subset. **Steps:** domain audit, selector in pipeline, CV tune, validate stability. **Parameters:** k , threshold, regularization. **Metrics:** task score + latency. **Pros/cons:** compact/interpretable / may lose interactions. **Best:** high-dimensional/noisy models.